DD: OSR-RMA-01-FLOW-EQP — Equipment Pickup at termination

Developer implementation guide: `DD_OSR-RMA-01-FLOW-EQP-DEV_IMPL_GUIDE-v1.0.md`.

1. Purpose

This rewritten DD is the developer-facing version of the operator BPMN flow `DD_OSR-RMA-01-FLOW-EQP-v1.0.md`.

Verdict: ■ New | **Wave:** 2 | **Group:** OSR (Operator Stock Registry) | **Version:** v1.0 | **Module:** Equipment Pickup at Termination Flow **Satellite of:** OSR-RMA-01 Equipment Swap & RMA Per-flow satellite for Equipment Pickup at subscription termination (`swap_kind=EQP_TERMINATION`). Owns the BPMN `EquipmentPickup_EQP_WIK.bpmn` + per-flow REST endpoint + per-flow Kafka payload shape + flow-specific Drools rules + worked end-to-end example. Reuses framework workers, messages, user tasks, history schema, and FOUNDATION_AUTH groups. v1.0 scope: KE WIK only. Handles both happy-path equipment recovery AND the EQR sub-state (customer refuses to return equipment) per wo_lifecycle.md Job Type catalog.

The goal of this rewrite is to make the DD easier for a mid-level developer to implement without losing the detailed source contract.

2. Implementation Scope

This DD should be implemented as part of the owning SOPHIX capability, not as an isolated service unless the deployment architecture explicitly separates that capability.

Implement this DD with these rules:

- Use the owning module APIs/repositories for authoritative writes.
- Use approved internal APIs/client wrappers when reading another module's data.
- Do not query another module's database tables directly from business flow code.
- Treat worker names as logical Camunda external-task topics, not separate deployments.
- One worker application may handle multiple topics, but metrics, retries, and logs must remain visible per topic.
- Emit success events only after the authoritative state commit succeeds.
- Use outbox publishing for Kafka events.

3. Runtime Metadata

Item	Value
Source file	`/Users/macbook/Downloads/Sophix_V3_BSS_DDs_MD_2026-05-27/08_osr_equipment/DD_OSR-RMA-01-FLOW-EQP-v1.0.md`
Version	`v1.0`
DD type	`operator BPMN flow`
BPMN/process keys	Not explicitly defined
Drools packages	Not explicitly defined

4. Runtime Contracts To Implement

4.1 APIs

- `POST /api/billable-events`
- `POST /api/contractor-slot-commitments`
- `POST /api/equipment-instances/inst_01HZ_ONT_KAMAU/lifecycle-events`
- `POST /api/swap-requests/eqp`
- `POST /engine-rest/task/{taskId}/complete`

4.2 Tables

- No CREATE TABLE block is explicitly declared in this DD.

For each table in this DD, implement the schema with:

- a short table comment explaining what the table is for;
- column comments or migration notes explaining each field;
- constraints and indexes from the DD;
- seed data where the DD provides operator-specific sample rows.

4.3 Worker Topics

- `bil01-charge-pre-authorize`
- `contractor-slot-reserve`
- `contractor-stock-route`
- `instance-state-transition`
- `swap-dispatchers`

Worker-topic guidance:

- A BPMN service task uses the topic.
- A worker application subscribes to the topic.
- The topic handler validates input variables, calls the owning module/API, writes outputs back to Camunda, and records metrics.
- Do not treat the topic string as a shell command or Kubernetes deployment name.

4.4 Events

- No domain events are explicitly declared in this DD.

Event guidance:

- Write events through the transactional outbox.
- Include operation id, aggregate id, operator code, actor, and correlation data.
- Consumers should be able to rebuild downstream state without querying workflow internals.

5. Developer Implementation Checklist

1. Add or verify database migrations and seed rows.
2. Add or verify config rows for the operator/module.
3. Implement request/command DTOs and validation.
4. Implement idempotency and in-flight operation checks where the DD requires them.
5. Implement Drools fact mapping and package deployment where rules are used.
6. Implement BPMN process, service tasks, gateways, timers, and message catches where the DD is a flow.
7. Implement worker-topic handlers using owning module APIs.
8. Implement compensation paths and manual recovery paths.
9. Emit success/rejected events through the outbox.
10. Add smoke tests for happy path, validation failure, dependency failure, and retry/idempotency.

6. Infrastructure And AWS Notes

Deploy this DD with the existing SOPHIX platform components unless the owning module requires isolation:

Concern	Recommendation
API/runtime	Run inside the owning module deployment or existing workflow API pods.
Workers	Consolidate low-volume topics into shared worker apps; keep per-topic metrics.
Database	Use the owning module PostgreSQL schema and migration pipeline.
Kafka	Use the foundation Kafka/MSK setup and transactional outbox.
Camunda	Deploy BPMN files through the workflow deployment pipeline.
Drools	Deploy KJAR/rule artifacts through the approved rules pipeline.
Secrets	Use the platform secret manager; on AWS prefer Secrets Manager/Parameter Store with IRSA.
Observability	Alert on stuck operations, failed workers, outbox backlog, and external dependency failures.

7. Original DD Detail Preserved

The following section preserves the detailed source contract for implementation and review.

Verdict: ■ New | **Wave:** 2 | **Group:** OSR (Operator Stock Registry) | **Version:** v1.0 | **Module:** Equipment Pickup at Termination Flow

Satellite of: OSR-RMA-01 Equipment Swap & RMA

Per-flow satellite for Equipment Pickup at subscription termination (`swap_kind=EQP_TERMINATION`). Owns the BPMN `EquipmentPickup_EQP_WIK.bpmn` + per-flow REST endpoint + per-flow Kafka payload shape + flow-specific Drools rules + worked end-to-end example. Reuses framework workers, messages, user tasks, history schema, and FOUNDATION_AUTH groups.

v1.0 scope: KE WIK only. Handles both happy-path equipment recovery AND the EQR sub-state (customer refuses to return equipment) per wo_lifecycle.md Job Type catalog.

1. What this flow is

When a customer terminates their subscription, all CPE equipment currently bound to that account (ONTs, modems, MTAs, STBs, smart cards) must be physically retrieved by a contractor. The termination workflow (`SUB-WF-TERMINATE-01`) creates one EQP swap-request per bound instance and waits for each to close before the termination itself can finalize (deposit refund or forfeiture, account closure).

This flow has two terminal paths:

- **Happy path (`COMPLETED`):** Tech visits, customer hands over equipment, instance transitions to `RECOVERED_BY_CONTRACTOR`, deposit refund is queued via BIL-01 + PLM-CFG-05 adjustment.
- **EQR sub-state (`COMPLETED_WITHOUT_RECOVERY`):** Customer refuses to return the equipment (e.g., "I paid for the CPE, it's mine"). Tech documents on WO; swap-request closes without recovery; instance stays `IN_FIELD_DEFECTIVE`; BIL-01 forfeits the deposit (deposit becomes operator revenue, not refunded).

The flow is *always* field-visit (no remote-EQP path — you can't get the physical box back without a truck). It's *always* free for the customer (no charge to the customer for the visit itself; the only money flow is deposit refund OR deposit forfeiture).

What this flow does NOT do

- **Does not orchestrate the subscription closure.** That's SUB-WF-TERMINATE-01. This flow is a child workflow: TERMINATE creates EQP swap-requests, EQP runs to completion, TERMINATE consumes the EQP completion events and proceeds with its own steps (final invoice, account closure).
- **Does not handle non-termination equipment retrievals.** If you want to recover equipment from a customer who's still active (e.g., end-of-life SKU forced replacement), use EQU (Equipment Upgrade) or HFC/GPON swap with appropriate reason codes.
- **Does not handle the deposit refund computation.** That's BIL-01 + PLM-CFG-05 (Adjustment Catalog). This flow signals "ready for refund" or "forfeit"; BIL-01 owns the financial path.

- **Does not transition the source instance to `DECOMMISSIONED` in EQR.** The instance stays `IN_FIELD_DEFECTIVE` (the customer has it, it's not in the operator's hands). A separate ops decision later may DECOMMISSION it (write-off after N months in IN_FIELD_DEFECTIVE with no contact).
- **Does not update HomePass `network_path`.** Termination doesn't remove the ONT from HomePass; that's a separate concern (HomePass status may transition to NSN or another non-ACT status by subscription closure workflow — not this flow's responsibility).

Glossary additions (specific to this flow)

Term	Meaning
EQP (Equipment Pickup)	Job Type code in wo_lifecycle.md for the WO that retrieves equipment at termination. WO is `wo_kind=EQUIPMENT_PICKUP`.
EQR (Equipment Recovery failed)	Sub-state of EQP. Per wo_lifecycle.md: "client paid for CPE & refuses return." Tech documents on WO finalization with outcome `EQR_FAILED_RETURN`.
Deposit refund	The amount the customer paid at install as a CPE security deposit. Refunded on successful EQP recovery via BIL-01 charge code `DEPOSIT_REFUND` + PLM-CFG-05 adjustment type `DEPOSIT`.
Deposit forfeiture	The deposit is NOT refunded; instead it becomes operator revenue (write-off prevented). Applied on EQR. Posted via BIL-01 charge code `DEPOSIT_FORFEITURE`.
Multi-device termination	When one customer has multiple CPE instances (e.g., triple-play: ONT + STB + smart card), each instance gets its own EQP swap-request. The TERMINATE workflow tracks all of them and only proceeds when all are terminal.

2. BPMN — `EquipmentPickup_EQP_WIK.bpmn`

2.1 Process shape

```
[start]
  ↓
[serviceTask: eligibility-check]
  ↓
[serviceTask: contractor-slot-reserve]
  ↓
[serviceTask: wo-01-create] (kind=EQUIPMENT_PICKUP)
  ↓
[userTask: swap-dispatch-slot-selection] (group=swap-dispatchers)
  ↓
[serviceTask: notification-send] (template: osr.rma.eqp.appointment-proposed)
  ↓
[intermediateCatch: customer-appointment-accepted] (boundary: customer-appointment-rejected → retry 3x → FAIL)
  ↓
[intermediateCatch: wo-finalized]
  ↓
[gateway: outcome]
  ■■■ SUCCESS → [userTask: tech-confirm-recovery-or-eqr] (recoveryOutcome=RECOVERED)
  ■               ↓
  ■             [serviceTask: instance-state-transition] (source → RECOVERED_BY_CONTRACTOR)
  ■               ↓
  ■             [serviceTask: contractor-stock-route]
  ■               ↓
  ■             [serviceTask: bil01-deposit-refund-queue] (charge_code=DEPOSIT_REFUND)
  ■               ↓
  ■             [end: COMPLETED]
  ■■■ EQR_FAILED_RETURN → [userTask: tech-confirm-recovery-or-eqr] (recoveryOutcome=CUSTOMER_REFUSED_RETURN)
  ■                         ↓
  ■                       [serviceTask: bil01-deposit-forfeiture-queue] (charge_code=DEPOSIT_FORFEITURE)
  ■                         ↓
  ■                       [serviceTask: notification-send] (template: osr.rma.eqp.eqr-deposit-forfeited)
  ■                         ↓
  ■                       [end: COMPLETED_WITHOUT_RECOVERY]
  ■■■ CUSTOMER_REFUSED_APPOINTMENT_3X → [end: FAILED]
```

2.2 BPMN XML — key snippets

Tech outcome decision gateway:

```
<bpmn:userTask id="Task_TechConfirmRecoveryOrEqr" name="Tech: confirm recovery or EQR"
               camunda:candidateGroups="field-techs">
  <bpmn:extensionElements>
    <camunda:formData>
```

```
        <camunda:formField id="recoveryOutcome" label="Outcome" type="enum">
          <camunda:value id="RECOVERED" name="Equipment recovered" />
          <camunda:value id="CUSTOMER_REFUSED_RETURN" name="Customer refused to return equipment (EQR)" />
        </camunda:formField>
        <camunda:formField id="notes" label="Notes" type="string" />
      </camunda:formData>
    </bpmn:extensionElements>
  </bpmn:userTask>

  <bpmn:exclusiveGateway id="Gateway_EqrSplit" name="Recovery outcome" />
  <bpmn:sequenceFlow id="Flow_Recovered" sourceRef="Gateway_EqrSplit" targetRef="Task_InstanceStateTransition_Recovered">
    <bpmn:conditionExpression>${recoveryOutcome == 'RECOVERED'}</bpmn:conditionExpression>
  </bpmn:sequenceFlow>
  <bpmn:sequenceFlow id="Flow_Eqr" sourceRef="Gateway_EqrSplit" targetRef="Task_DepositForfeitureQueue">
    <bpmn:conditionExpression>${recoveryOutcome == 'CUSTOMER_REFUSED_RETURN'}</bpmn:conditionExpression>
  </bpmn:sequenceFlow>
```

Note: the framework's `bil01-charge-pre-authorize` worker is REUSED for both `DEPOSIT_REFUND` and `DEPOSIT_FORFEITURE` posting. The same worker handles refund (negative-direction billable event) and forfeiture (positive-direction billable event) per BIL-01 + PLM-CFG-05 catalog semantics. We don't introduce new workers here; the worker's payload carries the direction.

2.3 Per-operator BPMN composition

KE WIK ships `EquipmentPickup_EQP_WIK.bpmn`. WUG/WTZ/YASSN variants TBD pending operator-specific termination workflow workshops.

3. Flow_payload shape (EQP_TERMINATION)

```json
{
  "depositHeld":             2500.00,
  "depositRefundEligible":  true,
  "recoveryRequired":       true,
  "terminationReason":      "CUSTOMER_REQUESTED",
  "depositForfeited":       false,
  "depositRefundAmount":    null,
  "depositForfeitureAmount": null
}
```

Field meanings:

Field	Sourced from	Purpose
`depositHeld`	BIL-01 (queried at eligibility)	Amount currently held as deposit for this CPE on this account
`depositRefundEligible`	BIL-01 rule (eligibility-check)	TRUE iff customer's account is not in arrears, dunning, or write-off
`recoveryRequired`	Operator config (default TRUE)	Some operators may allow customer-keeps-CPE on certain SKU classes — out of v1.0 scope
`terminationReason`	SUB-WF-TERMINATE-01 (passed in)	Audit trail; influences template selection for customer notification
`depositForfeited`	Set at EQR transition	Boolean flag for downstream consumers
`depositRefundAmount`	Resolved on RECOVERED → COMPLETED transition	Materialized for audit
`depositForfeitureAmount`	Resolved on EQR → COMPLETED_WITHOUT_RECOVERY transition	Materialized for audit (typically = depositHeld)

4. Rules (EQP-specific)

Rule	Description
R-EQP-1	Caller MUST be SUB-WF-TERMINATE-01 (or a manual ops invocation by `swap-dispatchers` group with explicit reason). Eligibility worker rejects if `reasonCode != TERMINATION` OR `subscription_id` is NULL.
R-EQP-2	One EQP swap-request per bound instance. Multi-device customers (ONT + STB + smart card) get one EQP each. SUB-WF-TERMINATE-01 creates them in parallel; the termination workflow tracks all via the multi-instance subprocess pattern in Camunda.

Rule	Description
R-EQP-3	Source instance MUST be `IN_FIELD_ACTIVE` or `IN_FIELD_DEFECTIVE`. Eligibility rejects with `failureCode=NOT_IN_FIELD` otherwise.
R-EQP-4	No target instance. `equipment_swap_request.target_instance_id` MUST be NULL for EQP. Validated at creation.
R-EQP-5	EQP is NEVER chargeable to the customer for the visit. `chargeable=FALSE` always. The pay-first gate is skipped. (Deposit refund / forfeiture is a separate financial flow, not customer-chargeable.)
R-EQP-6	On RECOVERED outcome: deposit refund is queued via BIL-01 with `chargeCode=DEPOSIT_REFUND`, `amount=depositHeld`, `direction=REFUND_TO_CUSTOMER`. PLM-CFG-05 adjustment type = `DEPOSIT`.
R-EQP-7	On EQR outcome: deposit forfeiture is queued via BIL-01 with `chargeCode=DEPOSIT_FORFEITURE`, `amount=depositHeld`, `direction=RETAIN`. The source instance stays `IN_FIELD_DEFECTIVE`; it is NOT transitioned to `RECOVERED_BY_CONTRACTOR` or `DECOMMISSIONED` by this flow. A later ops decision may DECOMMISSION it.
R-EQP-8	Customer-appointment retry: up to 3 attempts before transitioning to `FAILED`. Each rejection triggers a new appointment proposal via NOT-01 with adjusted slot. On 3rd rejection, swap-request → `FAILED` with `transition_reason=CUSTOMER_REFUSED_APPOINTMENT_3X`. SUB-WF-TERMINATE-01 escalates to ops (manual scheduling).
R-EQP-9	When SUB-WF-TERMINATE-01 cancels a pending termination (e.g., customer changes mind, retains subscription), it MUST cancel any open EQP swap-requests via the framework's cancel API. Open EQPs become orphaned otherwise. The cancel call releases the slot commitment (R-OSR-RMA-7).
R-EQP-10	EQR is the ONLY way to reach `COMPLETED_WITHOUT_RECOVERY`. Other failure modes (truck-roll cancelled, tech no-show, customer not home 3x) all route to `FAILED` (not COMPLETED_WITHOUT_RECOVERY). EQR is specifically "customer was there, refused to return."

5. REST API

```
POST /api/swap-requests/eqp HTTP/1.1
Authorization: Bearer <sub-wf-terminate-jwt>
Content-Type: application/json

{
  "operatorCode":        "WIK",
  "accountId":           "acct_01HZ_KAMAU",
  "subscriptionId":      "sub_01HZ_KAMAU_FIBER",
  "sourceInstanceId":    "inst_01HZ_ONT_KAMAU",
  "reasonCode":          "TERMINATION",
  "initiatedByUserId":   "sub-wf-terminate-system",
  "flowPayload": {
    "depositHeld":            2500.00,
    "depositRefundEligible":  true,
    "recoveryRequired":       true,
    "terminationReason":      "CUSTOMER_REQUESTED"
  },
  "notes": "Customer terminating fiber subscription effective 2026-06-30; equipment pickup required"
}

→ 201 Created
{
  "swapRequestId":       "swap_01HZ_KAMAU_TERM",
  "operatorCode":        "WIK",
  "swapKind":            "EQP_TERMINATION",
  "currentState":        "ELIGIBILITY_CHECK",
  "accountId":           "acct_01HZ_KAMAU",
  "subscriptionId":      "sub_01HZ_KAMAU_FIBER",
  "sourceInstanceId":    "inst_01HZ_ONT_KAMAU",
  "targetInstanceId":    null,
  "reasonCode":          "TERMINATION",
  "initiatedByUserId":   "sub-wf-terminate-system",
  "initiatedAt":         "2026-06-20T10:00:00Z",
  "bpmnProcessInstanceId": "camunda-pi-9c5d2e0f"
}
```

6. Kafka events emitted

6.1 `SwapRequestCompleted` (happy path)

```
{
  "eventType":   "SwapRequestCompleted",
  "aggregateId": "swap_01HZ_KAMAU_TERM",
  "timestamp":   "2026-06-30T14:25:00.000Z",
  "payload": {
    "operatorCode":          "WIK",
    "swapRequestId":         "swap_01HZ_KAMAU_TERM",
    "swapKind":              "EQP_TERMINATION",
    "accountId":             "acct_01HZ_KAMAU",
    "subscriptionId":        "sub_01HZ_KAMAU_FIBER",
    "sourceInstanceId":      "inst_01HZ_ONT_KAMAU",
    "targetInstanceId":      null,
    "recoveryContractorId": "contractor_ke_nairobi_a",
    "woId":                  "wo_01HZ_KAMAU_EQP",
    "chargeCode":            "DEPOSIT_REFUND",
    "depositRefundAmount":  2500.00,
    "closedAt":              "2026-06-30T14:25:00+03:00",
    "completionPath":        "RECOVERED"
  }
}
```

6.2 `SwapRequestCompletedWithoutRecovery` (EQR)

```
{
  "eventType":   "SwapRequestCompletedWithoutRecovery",
  "aggregateId": "swap_01HZ_OTIENO_TERM_EQR",
  "timestamp":   "2026-06-30T15:10:00.000Z",
  "payload": {
    "operatorCode":              "WIK",
    "swapRequestId":             "swap_01HZ_OTIENO_TERM_EQR",
    "swapKind":                  "EQP_TERMINATION",
    "accountId":                 "acct_01HZ_OTIENO",
    "subscriptionId":            "sub_01HZ_OTIENO_TRIPLEPLAY",
    "sourceInstanceId":          "inst_01HZ_STB_OTIENO",
    "recoveryContractorId":      "contractor_ke_nairobi_a",
    "woId":                      "wo_01HZ_OTIENO_EQP",
    "chargeCode":                "DEPOSIT_FORFEITURE",
    "depositForfeitureAmount":  2500.00,
    "techNotes":                 "Customer states they paid for the STB and consider it their property; refused to return c
    "closedAt":                  "2026-06-30T15:10:00+03:00",
    "completionPath":            "EQR_FAILED_RETURN"
  }
}
```

7. Worked end-to-end example — Kamau's termination (happy path)

Customer: Kamau, WIK fiber single-play subscriber. ONT `inst_01HZ_ONT_KAMAU` at his premises. Account `acct_01HZ_KAMAU`. Subscription `sub_01HZ_KAMAU_FIBER` terminating effective 2026-06-30 (customer-requested, 10-day notice given on 2026-06-20). Deposit of KES 2,500 held. Account in good standing. Tech region resolves to `contractor_ke_nairobi_a` (Franchise A).

Day 0 (2026-06-20) — 10:00: SUB-WF-TERMINATE-01 advances and reaches the equipment-retrieval step. For each instance bound to Kamau's account, it creates an EQP swap-request:

```
POST /api/swap-requests/eqp
{"operatorCode":"WIK","accountId":"acct_01HZ_KAMAU","subscriptionId":"sub_01HZ_KAMAU_FIBER",
 "sourceInstanceId":"inst_01HZ_ONT_KAMAU","reasonCode":"TERMINATION",
 "initiatedByUserId":"sub-wf-terminate-system",
 "flowPayload":{"depositHeld":2500.00,"depositRefundEligible":true,"recoveryRequired":true,
               "terminationReason":"CUSTOMER_REQUESTED"}}
→ 201 swap_01HZ_KAMAU_TERM
```

BPMN starts. Eligibility passes (account active, instance IN_FIELD_ACTIVE, deposit confirmed). State advances directly to `AWAITING_SLOT` (no pay-first; EQP is never chargeable).

Day 0 — 10:01: Worker `contractor-slot-reserve` calls EM-02:

```
POST /api/contractor-slot-commitments
{"operatorCode":"WIK","accountId":"acct_01HZ_KAMAU","techRegionId":"tregion_ke_nrb_central",
 "serviceScope":"GPON","requiredSkills":["EQUIPMENT_PICKUP_GPON"],
 "windowStart":"2026-06-29T08:00:00+03:00","windowEnd":"2026-06-30T18:00:00+03:00"}
→ 201 {"slotCommitmentId":"slot_01HZ_KAMAU_EQP","contractorId":"contractor_ke_nairobi_a",
       "scheduledFor":"2026-06-30T14:00:00+03:00"}
```

Worker calls WO-01 → `wo_01HZ_KAMAU_EQP` created with `wo_kind=EQUIPMENT_PICKUP`. NOT-01 sends Kamau SMS proposing the slot. Kamau replies YES.

Day 10 (2026-06-30) — 14:05: Franchise A tech `tech_franchise_a_03` arrives. Kamau hands over the ONT. Tech opens app, completes user task:

```
POST /engine-rest/task/{taskId}/complete
{"variables":{"recoveryOutcome":{"value":"RECOVERED"},"notes":{"value":"ONT in good condition; returned in original box"}}
```

State advances `FIELD_VISIT_IN_PROGRESS` → `SOURCE_RECOVERED`. Worker `instance-state-transition`:

```
POST /api/equipment-instances/inst_01HZ_ONT_KAMAU/lifecycle-events
{"toState":"RECOVERED_BY_CONTRACTOR","toLocationId":"WIK-VAN-contractor_ke_nairobi_a",
 "reasonCode":"CONTRACTOR_RECOVERED_FROM_FIELD","contractorId":"contractor_ke_nairobi_a",
 "originRefKind":"swap_request","originRefId":"swap_01HZ_KAMAU_TERM",
 "actorUserId":"tech_franchise_a_03","expectedEventSequence":5}
→ 201 evt_01HZ_KAMAU_RECOV
```

Worker `contractor-stock-route` writes the stock movement. ONT is now in Franchise A's van.

Day 10 — 14:20: Worker `bil01-charge-pre-authorize` fires for deposit refund (reused worker, refund-direction payload):

```
POST /api/billable-events
{"accountId":"acct_01HZ_KAMAU","chargeCode":"DEPOSIT_REFUND","amount":2500.00,
 "direction":"REFUND_TO_CUSTOMER","contextRef":{"kind":"swap_request","id":"swap_01HZ_KAMAU_TERM"},
 "adjustmentTypeRef":"DEPOSIT"}
→ 201 {"billableEventId":"bevt_01HZ_KAMAU_REFUND","status":"QUEUED"}
```

State `PROVISIONING_IN_PROGRESS` → `COMPLETED`. Kafka emits `SwapRequestCompleted` with `completionPath=RECOVERED`. SUB-WF-TERMINATE-01 catches the event, marks this EQP done, proceeds with its own next step (final invoice, account closure).

Day 10 — 14:25: NOT-01 sends Kamau: *"Hi Kamau, we've received your equipment. Your KES 2,500 deposit will be refunded to your registered M-Pesa within 7 business days. Thank you for being a WIK customer."*

8. Worked end-to-end example — Otieno's termination (EQR)

Customer: Otieno, WIK HFC triple-play subscriber. STB `inst_01HZ_STB_OTIENO` (smart card 7676543210). Terminating. Deposit KES 2,500.

Days 0–10 identical to Kamau's setup. Tech arrives Day 10 at 15:00.

Day 10 — 15:05: Tech asks Otieno for the STB. Otieno: "I paid 5,000 shillings for that decoder. It's mine. I'm not giving it back." Tech explains T&Cs (deposit is for the box, not purchase); Otieno remains firm. Tech submits user task:

```
POST /engine-rest/task/{taskId}/complete
{"variables":{"recoveryOutcome":{"value":"CUSTOMER_REFUSED_RETURN"},
 "notes":{"value":"Customer states they paid for the STB and consider it their property; refused to return despite explana
```

Gateway routes to EQR branch. Worker `bil01-charge-pre-authorize` fires for deposit forfeiture (reused worker, retain-direction payload):

```
POST /api/billable-events
{"accountId":"acct_01HZ_OTIENO","chargeCode":"DEPOSIT_FORFEITURE","amount":2500.00,
 "direction":"RETAIN","contextRef":{"kind":"swap_request","id":"swap_01HZ_OTIENO_TERM_EQR"},
 "adjustmentTypeRef":"DEPOSIT"}
→ 201 {"billableEventId":"bevt_01HZ_OTIENO_FORFEIT","status":"QUEUED"}
```

Source instance NOT transitioned by this flow (R-EQP-7): `inst_01HZ_STB_OTIENO` stays in `IN_FIELD_ACTIVE` until a separate ops decision later transitions it to `IN_FIELD_DEFECTIVE` (subscription is terminated; instance is effectively orphaned) or `DECOMMISSIONED` (write-off).

Actually — clarification: per R-EQP-7, the instance stays `IN_FIELD_DEFECTIVE` (not ACTIVE) because the subscription is terminated and the device is logically disconnected from active service even if physically still at the customer's premises. The lifecycle transition is performed by SUB-WF-TERMINATE-01 (not this flow) at subscription closure.

State `FIELD_VISIT_IN_PROGRESS` → `COMPLETED_WITHOUT_RECOVERY`. Kafka emits `SwapRequestCompletedWithoutRecovery` with `completionPath=EQR_FAILED_RETURN`. SUB-WF-TERMINATE-01 catches; marks this EQP forfeited; proceeds with final invoice (deposit not refunded).

Day 10 — 15:10: NOT-01 sends Otieno: *"Hi Otieno, per our terms of service, the equipment deposit (KES 2,500) is retained as the equipment was not returned. If you change your mind, contact our office within 30 days. Your subscription is closed."*

9. Workshop sources

- **wo_lifecycle.md Job Type catalog** — EQP (Equipment Pickup) and EQR (Equipment Recovery failed: "client paid for CPE & refuses return"). Drives the dual terminal paths in §2 BPMN and R-EQP-7 / R-EQP-10.
- **Workshop 2026-04-30 (Provisioning deep dive — Service Termination)** — Equipment dismantle = separate retrieval WO if customer doesn't bring CPE back. For both GPON and HFC, an agent contractor is sent on site. Refund processed via Invoice Adjustments. Drives R-EQP-1 (caller is SUB-WF-TERMINATE-01), R-EQP-2 (one per device), and R-EQP-6 (BIL-01 refund flow via PLM-CFG-05 DEPOSIT adjustment type).
- **Workshop 2026-04-30 (Customer Management deep dive — Equipment routing gap)** — 40+ stale equipment items. Drives the `contractor-stock-route` worker invocation at Day 10 14:20.
- **DD_PLM-CFG-05 Adjustment Type Catalog** — `DEPOSIT` adjustment type. Used in §6 Kafka payloads + §7/§8 BIL-01 calls.
- **BP-08 Equipment Swap (RMA) GAP analysis** — Equipment pickup at termination is part of the Wananchi target scope; this satellite addresses it.